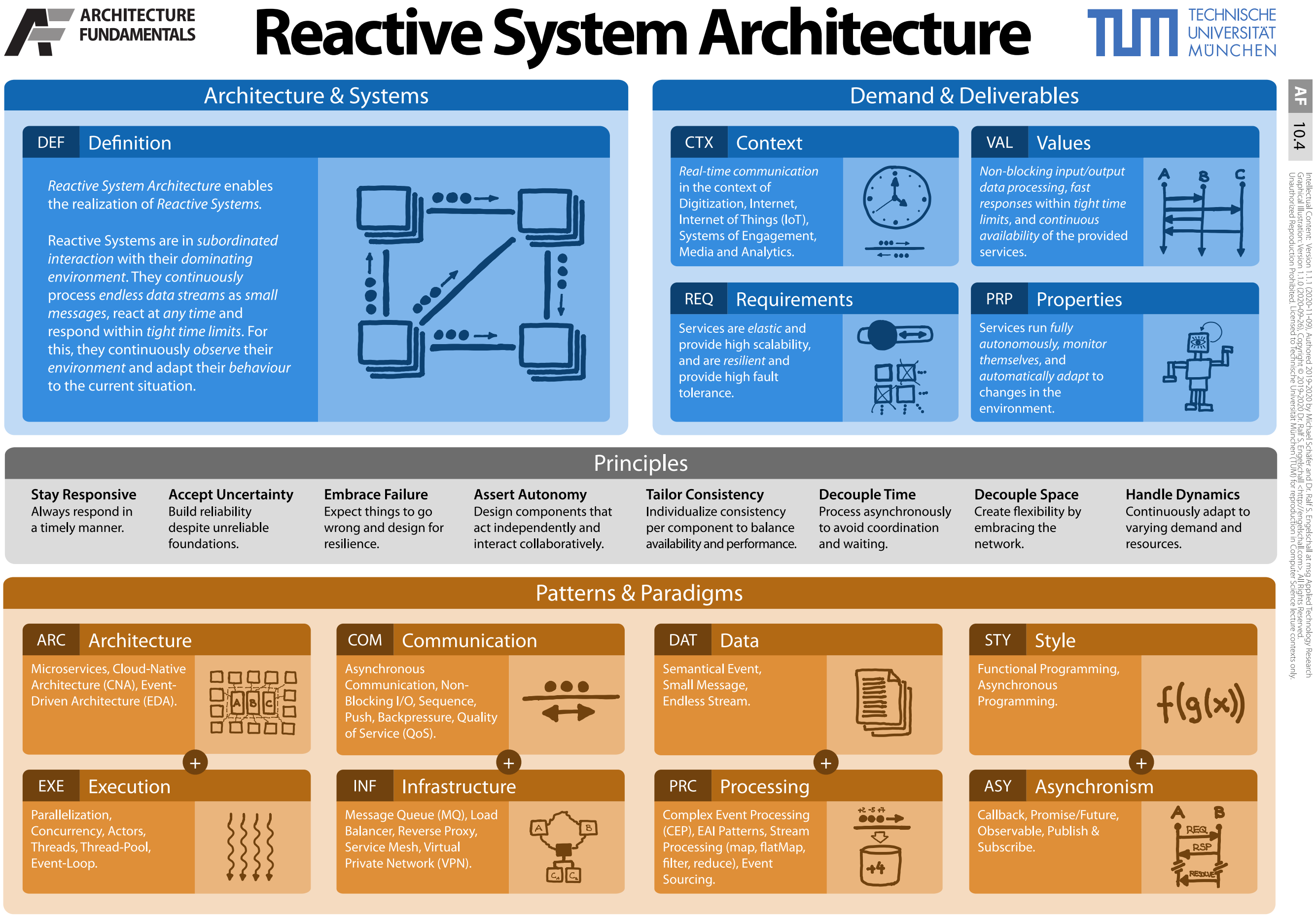
TECHNISCHE
UNIVERSITÄT
MÜNCHEN

Software Engineering in Industrial Practice (SEIP)

Dr. Ralf S. Engelschall



ARCHITECTURE FUNDAMENTALS

Reactive System Architecture

TECHNISCHE UNIVERSITÄT MÜNCHEN

AF 10.4

Architecture & Systems

DEF Definition

Reactive System Architecture enables the realization of *Reactive Systems.*

Reactive Systems are in *subordinated interaction* with their *dominating environment.* They *continuously* process *endless data streams* as *small messages*, react at *any time* and respond within *tight time limits.* For this, they continuously *observe* their *environment* and adapt their *behaviour* to the current situation.

Demand & Deliverables

CTX Context

Real-time communication in the context of Digitization, Internet, Internet of Things (IoT), Systems of Engagement, Media and Analytics.

VAL Values

Non-blocking input/output data processing, fast responses within *tight time limits*, and *continuous availability* of the provided services.

REQ Requirements

Services are *elastic* and provide high scalability, and are *resilient* and provide high fault tolerance.

PRP Properties

Services run *fully autonomously, monitor themselves*, and *automatically adapt* to changes in the environment.

Principles

Principle	Description
Stay Responsive	Always respond in a timely manner.
Accept Uncertainty	Build reliability despite unreliable foundations.
Embrace Failure	Expect things to go wrong and design for resilience.
Assert Autonomy	Design components that act independently and interact collaboratively.
Tailor Consistency	Individualize consistency per component to balance availability and performance.
Decouple Time	Process asynchronously to avoid coordination and waiting.
Decouple Space	Create flexibility by embracing the network.
Handle Dynamics	Continuously adapt to varying demand and resources.

Patterns & Paradigms

ARC Architecture

Microservices, Cloud-Native Architecture (CNA), Event-Driven Architecture (EDA).

EXE Execution

Parallelization, Concurrency, Actors, Threads, Thread-Pool, Event-Loop.

COM Communication

Asynchronous Communication, Non-Blocking I/O, Sequence, Push, Backpressure, Quality of Service (QoS).

INF Infrastructure

Message Queue (MQ), Load Balancer, Reverse Proxy, Service Mesh, Virtual Private Network (VPN).

DAT Data

Semantical Event, Small Message, Endless Stream.

PRC Processing

Complex Event Processing (CEP), EAI Patterns, Stream Processing (map, flatMap, filter, reduce), Event Sourcing.

STY Style

Functional Programming, Asynchronous Programming.

ASY Asynchronism

Callback, Promise/Future, Observable, Publish & Subscribe.

Intellectual Content: Version 1.1.1 (2020-11-09), Authored 2019-2020 by Michael Schäfer and Dr. Ralf S. Engelschall at msg Applied Technology Research
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

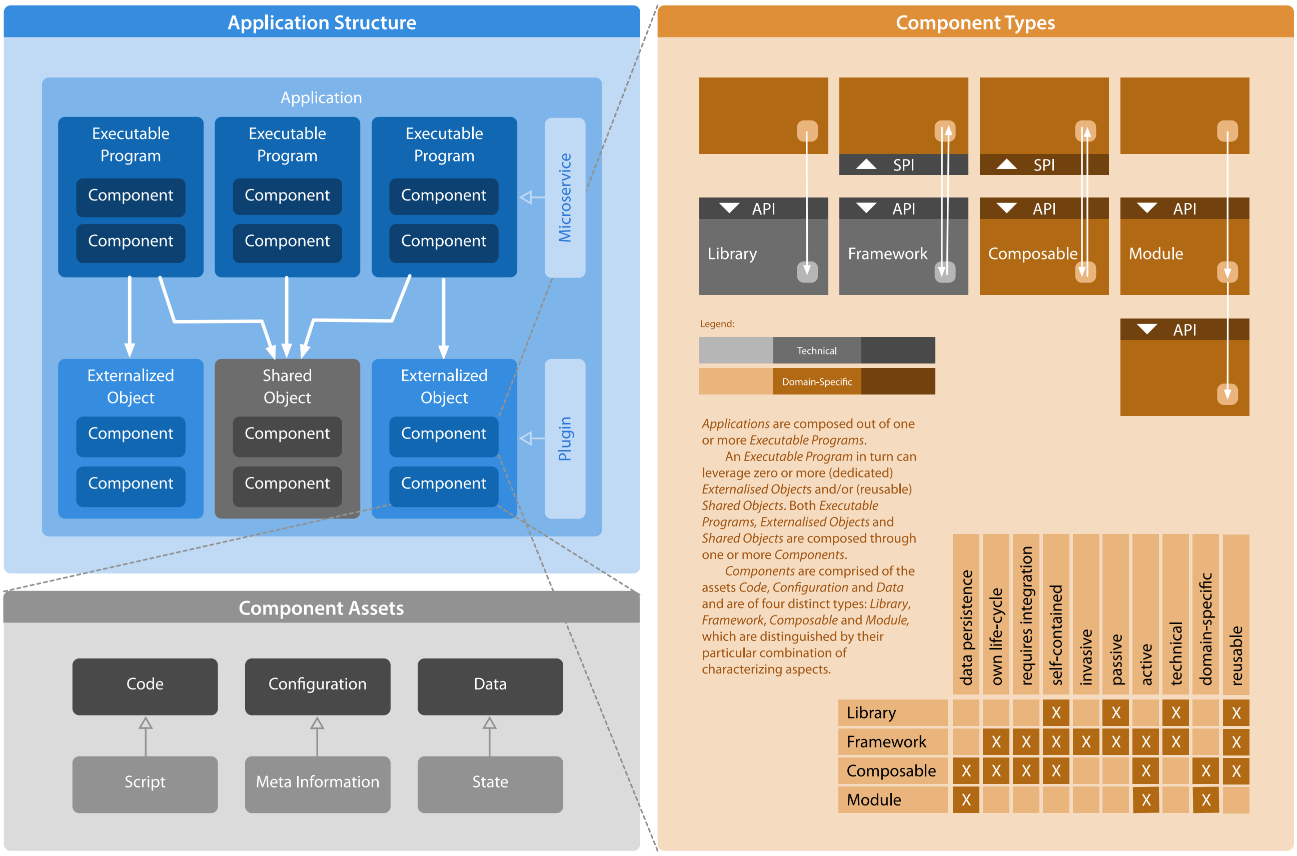
Reactive System Architecture erlaubt die Realisierung von **Reactive Systems**. Reactive Systems sind in **untergeordneter Interaktion** mit ihrer **dominierenden** Umgebung. Sie **verarbeiten kontinuierlich endlose Datenströme** in Form von **kleinen Nachrichten**, reagieren zu **jeder Zeit** und antworten innerhalb **enger Zeitgrenzen**. Dazu **beobachten** sie kontinuierlich ihre **Umgebung** und **passen** ihr **Verhalten** an die aktuelle Situation an.

Reactive Systems werden primär im Kontext von Echtzeit-Kommunikation verwendet, wo Dienste bereitgestellt werden, die **elastisch** (**elastic**) sein und hohe Skalierbarkeit anbieten müssen und die **verlässlich** (**reliable**) sein und hohe Fehlertoleranz anbieten müssen.

Fragen

- ? Welche zwei wesentlichen Anforderungen erfüllen Reactive Systems?
- ? Was charakterisiert Reactive Systems in Bezug auf ihre Datenverarbeitung?

Application Composition


Application Structure
Application
Executable Program
Component
Component
Executable Program
Component
Component
Executable Program
Component
Component
Microservice
Externalized Object
Component
Component
Shared Object
Component
Component
Externalized Object
Component
Component
Plugin
Component Assets
Code
Configuration
Data
Script
Meta Information
State
Component Types
SPI
SPI
API
API
API
API
Library
Framework
Composable
Module
API
Legend:
Technical
Domain-Specific

Applications are composed out of one or more *Executable Programs*.

An *Executable Program* in turn can leverage zero or more (dedicated) *Externalised Objects* and/or (reusable) *Shared Objects*. Both *Executable Programs, Externalised Objects* and *Shared Objects* are composed through one or more *Components*.

Components are comprised of the assets *Code, Configuration* and *Data* and are of four distinct types: *Library, Framework, Composable* and *Module,* which are distinguished by their particular combination of characterizing aspects.

	data persistence	own life-cycle	requires integration	self-contained	invasive	passive	active	technical	domain-specific	reusable
Library				X		X		X		X
Framework		X	X	X	X	X	X	X		X
Composable	X	X	X	X			X		X	X
Module	X						X		X	

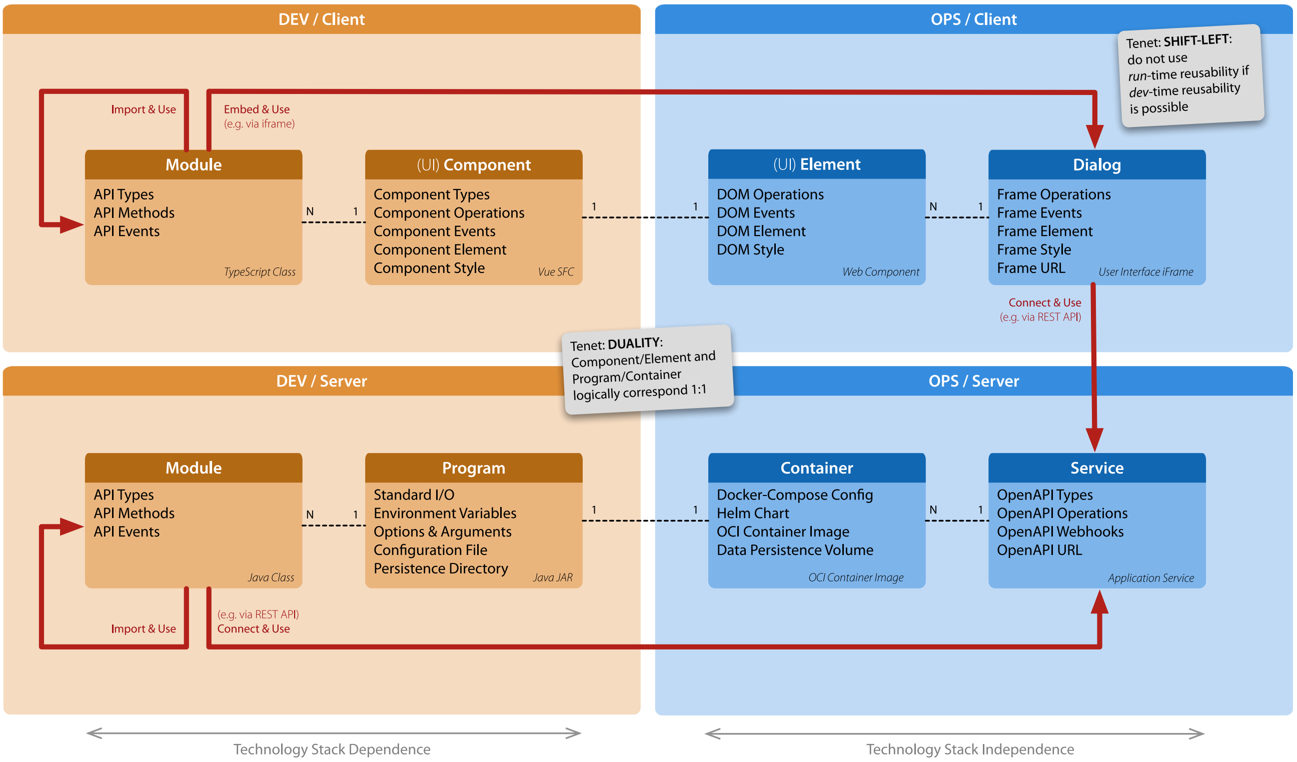
Anwendungen werden aus einem oder mehreren Executable Programs zusammengesetzt. Ein Executable Program kann seinerseits null oder mehrere (dedizierte) Externalized Objects und/oder (wiederverwendbare) Shared Objects nutzen. Sowohl Executable Programs als auch Externalized Objects und Shared Objects bestehen aus einer oder mehreren Components. In einer Microservice Architecture werden die Executable Programs als Microservices bezeichnet. In einer Plugin Architecture werden die Externalized Objects als Plugins bezeichnet.

Es gibt vier verschiedene Typen von Components: Library (Bibliothek), Framework, Composable und Module (Modul). Sie lassen sich durch ihre besondere Kombination von charakteristischen Aspekten unterscheiden. Am wichtigsten ist, ob sie ein Application Programming Interface (API) für den Konsumenten der Component bereitstellen und/oder ob sie verlangen, dass der Konsument der Component eine Art von Service Provider Interface (SPI) erfüllen muss.

Fragen

❓ Was ist der wesentliche Unterschied zwischen einer Library und einem Framework?

Functional Reusability



DEV / Client

Import & Use

Embed & Use
(e.g. via iframe)

Module
API Types
API Methods
API Events
TypeScript Class

N — 1

(UI) Component
Component Types
Component Operations
Component Events
Component Element
Component Style
Vue SFC

1 — 1

OPS / Client

Tenet: **SHIFT-LEFT:**
do not use
run-time reusability if
dev-time reusability
is possible

(UI) Element
DOM Operations
DOM Events
DOM Element
DOM Style
Web Component

N — 1

Dialog
Frame Operations
Frame Events
Frame Element
Frame Style
Frame URL
User Interface iFrame

Connect & Use
(e.g. via REST API)

Tenet: **DUALITY:**
Component/Element and
Program/Container
logically correspond 1:1

DEV / Server

Module
API Types
API Methods
API Events
Java Class

N — 1

Program
Standard I/O
Environment Variables
Options & Arguments
Configuration File
Persistence Directory
Java JAR

1 — 1

OPS / Server

Container
Docker-Compose Config
Helm Chart
OCI Container Image
Data Persistence Volume
OCI Container Image

N — 1

Service
OpenAPI Types
OpenAPI Operations
OpenAPI Webhooks
OpenAPI URL
Application Service

Import & Use

(e.g. via REST API)
Connect & Use

Technology Stack Dependence

Technology Stack Independence

Auf der Client-Seite unterscheiden wir die funktionellen Einheiten **Module** und **Component** während der Entwicklungszeit (DEV), und **Element** und **Dialog** während der Betriebszeit (OPS). Das **Module** besteht aus den regulären API-Typen, -Methoden und -Ereignissen. Die **Component** besteht aus den auf der Webtechnologie basierenden Komponenten-Typen (JS), -Operationen (JS), -Ereignissen (DOM), -Elementen (HTML) und -Auszeichnungen (CSS). Das **Element** besteht aus den auf der Webtechnologie basierenden DOM-Operationen (JS), -Ereignissen (DOM), -Elementen (HTML) und -Auszeichnungen (CSS). Der **Dialog** ist ein Benutzeroberflächenrahmen mit Operationen (JS), Ereignissen (DOM), Element (HTML), Auszeichnung (CSS) und URL.

Auf der Client-Seite können Sie Funktionen in einem **Module** wiederverwenden, indem Sie entweder ein anderes **Module** importieren (DEV) oder den Rahmen eines ganzen **Dialogs** einbetten (OPS).

Auf der Server-Seite unterscheiden wir die funktionellen Einheiten **Module** und **Program** während der Entwicklungszeit (DEV), und **Container** und **Service** während der Betriebszeit (OPS). Das **Module** besteht aus den regulären API-Typen, -Methoden und -Ereignissen. Das **Program** besteht aus Standard-I/O, Umgebungsvariablen, Optionen und Argumenten, Konfigurationsdatei und Persistenzverzeichnis. Der **Container** besteht aus Docker-Compose Config und/oder Helm Chart, OCI Container Image und Data Persistence Volume. Der **Service** besteht aus OpenAPI-Typen, -Operationen, -Webhooks und -URL.

Auf der Server-Seite können Sie Funktionen in einem **Module** wiederverwenden, indem Sie entweder ein anderes **Module** importieren (DEV) oder eine Verbindung zu einem **Service** herstellen (OPS).

Der Client-seitige **Dialog** verwendet ebenfalls die Funktionalität des Server-seitigen **Service**, indem er sich mit ihm verbindet.

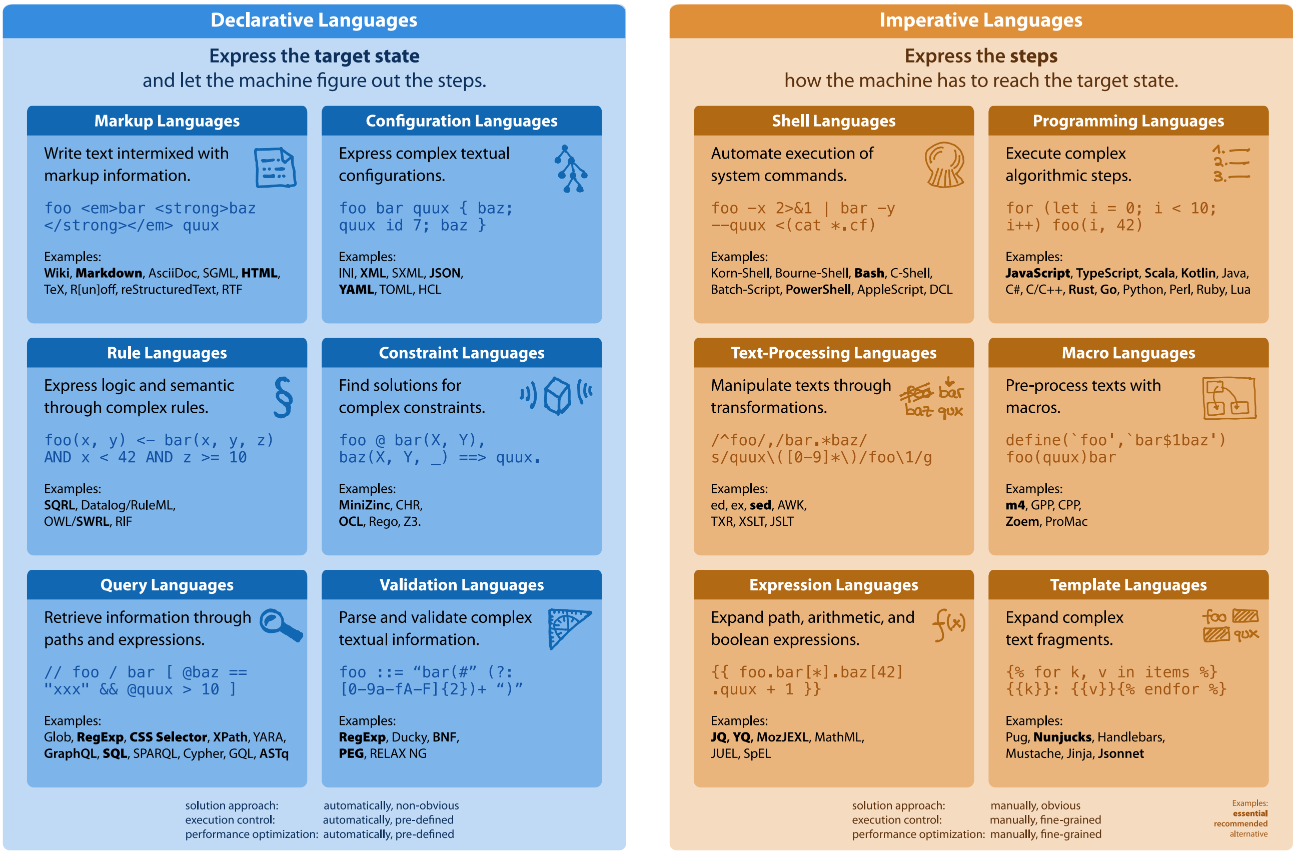
Fragen

❓ Wie kann man in einer Client/Server-Architektur Funktionalität **verwenden** bzw. **wiederverwenden**?

ARCHITECTURE FUNDAMENTALS

Formal Languages

TECHNISCHE UNIVERSITÄT MÜNCHEN



Declarative Languages

Express the target state
and let the machine figure out the steps.

Markup Languages

Write text intermixed with markup information.

```
foo <em>bar <strong>baz
</strong></em> quux
```

Examples:
Wiki, **Markdown**, AsciiDoc, SGML, **HTML**, TeX, R[un]off, reStructuredText, RTF

Configuration Languages

Express complex textual configurations.

```
foo bar quux { baz;
quux id 7; baz }
```

Examples:
INI, **XML**, SXML, **JSON**, **YAML**, TOML, HCL

Rule Languages

Express logic and semantic through complex rules.

```
foo(x, y) <- bar(x, y, z)
AND x < 42 AND z >= 10
```

Examples:
SQRL, Datalog/RuleML, OWL/**SWRL**, RIF

Constraint Languages

Find solutions for complex constraints.

```
foo @ bar(X, Y),
baz(X, Y, _) ==> quux.
```

Examples:
MiniZinc, CHR, **OCL**, Rego, Z3.

Query Languages

Retrieve information through paths and expressions.

```
// foo / bar [ @baz ==
"xxx" && @quux > 10 ]
```

Examples:
Glob, **RegExp**, **CSS Selector**, **XPath**, YARA, **GraphQL**, **SQL**, SPARQL, Cypher, GQL, **ASTq**

Validation Languages

Parse and validate complex textual information.

```
foo ::= “bar(#” (?:
[0-9a-fA-F]{2})+ “)”
```

Examples:
RegExp, Ducky, **BNF**, **PEG**, RELAX NG

solution approach: automatically, non-obvious
execution control: automatically, pre-defined
performance optimization: automatically, pre-defined

Imperative Languages

Express the steps
how the machine has to reach the target state.

Shell Languages

Automate execution of system commands.

```
foo -x 2>&1 | bar -y
--quux <(cat *.cf)
```

Examples:
Korn-Shell, Bourne-Shell, **Bash**, C-Shell, Batch-Script, **PowerShell**, AppleScript, DCL

Programming Languages

Execute complex algorithmic steps.

```
for (let i = 0; i < 10;
i++) foo(i, 42)
```

Examples:
JavaScript, **TypeScript**, **Scala**, **Kotlin**, Java, C#, C/C++, **Rust**, **Go**, Python, Perl, Ruby, Lua

Text-Processing Languages

Manipulate texts through transformations.

```
/^foo/,/bar.*baz/
s/quux\([0-9]*\)/foo\1/g
```

Examples:
ed, ex, **sed**, AWK, TXR, XSLT, JSLT

Macro Languages

Pre-process texts with macros.

```
define(`foo',`bar$1baz')
foo(quux)bar
```

Examples:
m4, GPP, CPP, **Zoem**, ProMac

Expression Languages

Expand path, arithmetic, and boolean expressions.

```
{{ foo.bar[*].baz[42]
.quux + 1 }}
```

Examples:
JQ, **YQ**, **MozJEXL**, MathML, JUEL, SpEL

Template Languages

Expand complex text fragments.

```
{% for k, v in items %}
{{k}}: {{v}}{% endfor %}
```

Examples:
Pug, **Nunjucks**, Handlebars, Mustache, Jinja, **Jsonnet**

solution approach: manually, obvious
execution control: manually, fine-grained
performance optimization: manually, fine-grained

Examples:
essential
recommended
alternative

Es gibt unzählige formale Sprachen (**Formal Languages**). In einem konkreten Technology Stack kommen üblicherweise fast ein Dutzend solcher Sprachen gleichzeitig zum Einsatz. Der Architekt muss diese deshalb sehr sorgfältig auswählen.

Die formalen Sprachen lassen sich zunächst in deklarative Sprachen (**Declarative Languages**) und imperative Sprachen (**Imperative Languages**) aufteilen. Erstere drücken einen Ziel-Zustand aus (“WAS”), letztere drücken dagegen den Weg dorthin aus (“WIE”).

Deklarative Ansätze sind imperativen Ansätzen meist vorzuziehen, da sie es der Implementierung (und nicht dem Programmierer) überlassen, den optimalen Weg zu finden. Außerdem erlauben sie inkrementelle Herangehensweisen, bei denen der nächste Schritt durch die konkrete Differenz zwischen dem aktuellen Ist-Zustand und dem gewünschten Ziel-Zustand abgeleitet wird. Dies ist besonders in sehr dynamischen und fehleranfälligen Umgebungen beim Wiederaufsetzen entscheidend.

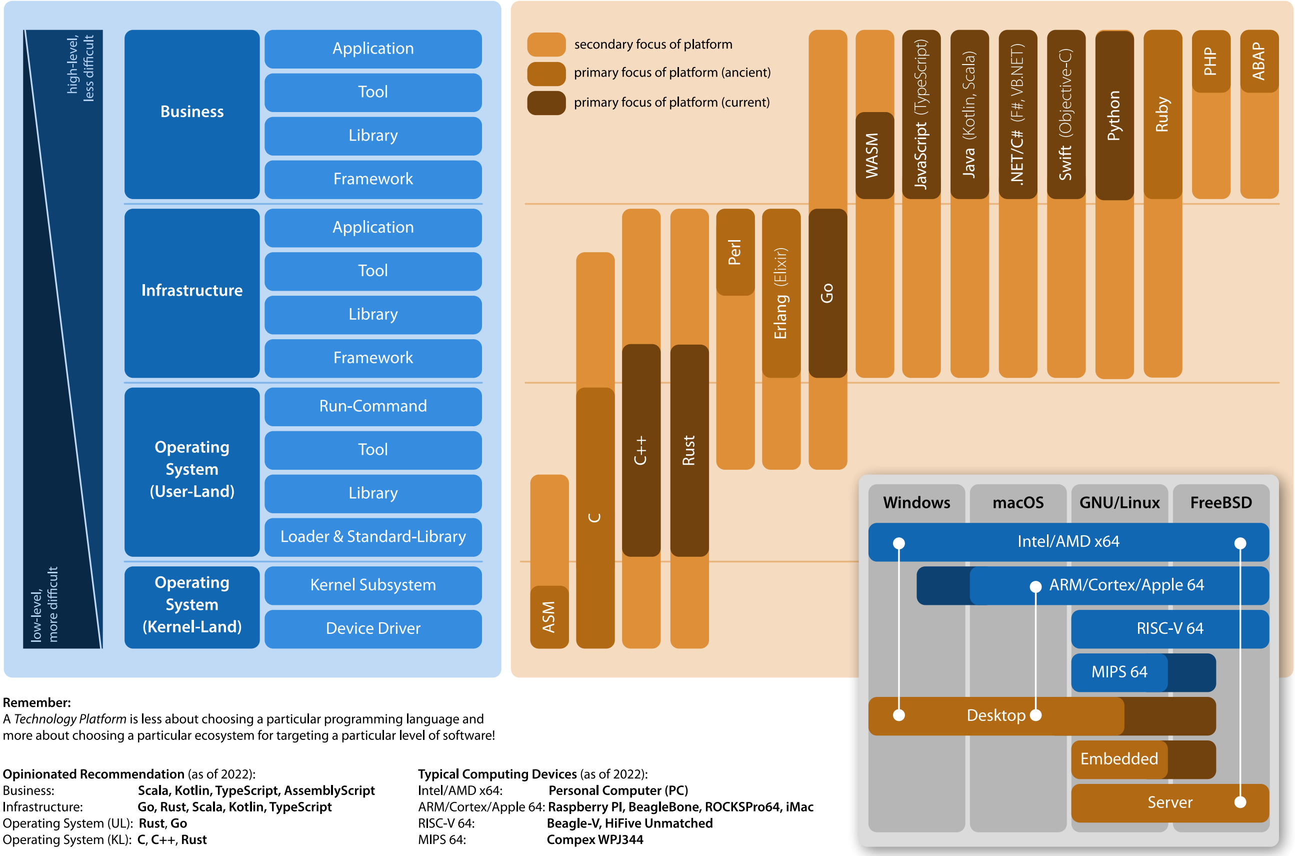
Fragen

? In welche zwei Klassen lassen sich formale Sprachen (**Formal Languages**) aufteilen?

ARCHITECTURE FUNDAMENTALS

Technology Platforms

TECHNISCHE UNIVERSITÄT MÜNCHEN


high-level, less difficult
low-level, more difficult
Business
Application
Tool
Library
Framework
Infrastructure
Application
Tool
Library
Framework
Operating System (User-Land)
Run-Command
Tool
Library
Loader & Standard-Library
Operating System (Kernel-Land)
Kernel Subsystem
Device Driver
secondary focus of platform
primary focus of platform (ancient)
primary focus of platform (current)
ASM
C
C++
Rust
Perl
Erlang (Elixir)
Go
WASM
JavaScript (TypeScript)
Java (Kotlin, Scala)
.NET/C# (F#, VB.NET)
Swift (Objective-C)
Python
Ruby
PHP
ABAP
Windows
macOS
GNU/Linux
FreeBSD
Intel/AMD x64
ARM/Cortex/Apple 64
RISC-V 64
MIPS 64
Desktop
Embedded
Server

Remember:
A *Technology Platform* is less about choosing a particular programming language and more about choosing a particular ecosystem for targeting a particular level of software!

Opinionated Recommendation (as of 2022):
Business: **Scala, Kotlin, TypeScript, AssemblyScript**
Infrastructure: **Go, Rust, Scala, Kotlin, TypeScript**
Operating System (UL): **Rust, Go**
Operating System (KL): **C, C++, Rust**

Typical Computing Devices (as of 2022):
Intel/AMD x64: **Personal Computer (PC)**
ARM/Cortex/Apple 64: **Raspberry PI, BeagleBone, ROCKSPro64, iMac**
RISC-V 64: **Beagle-V, HiFive Unmatched**
MIPS 64: **Compex WPJ344**

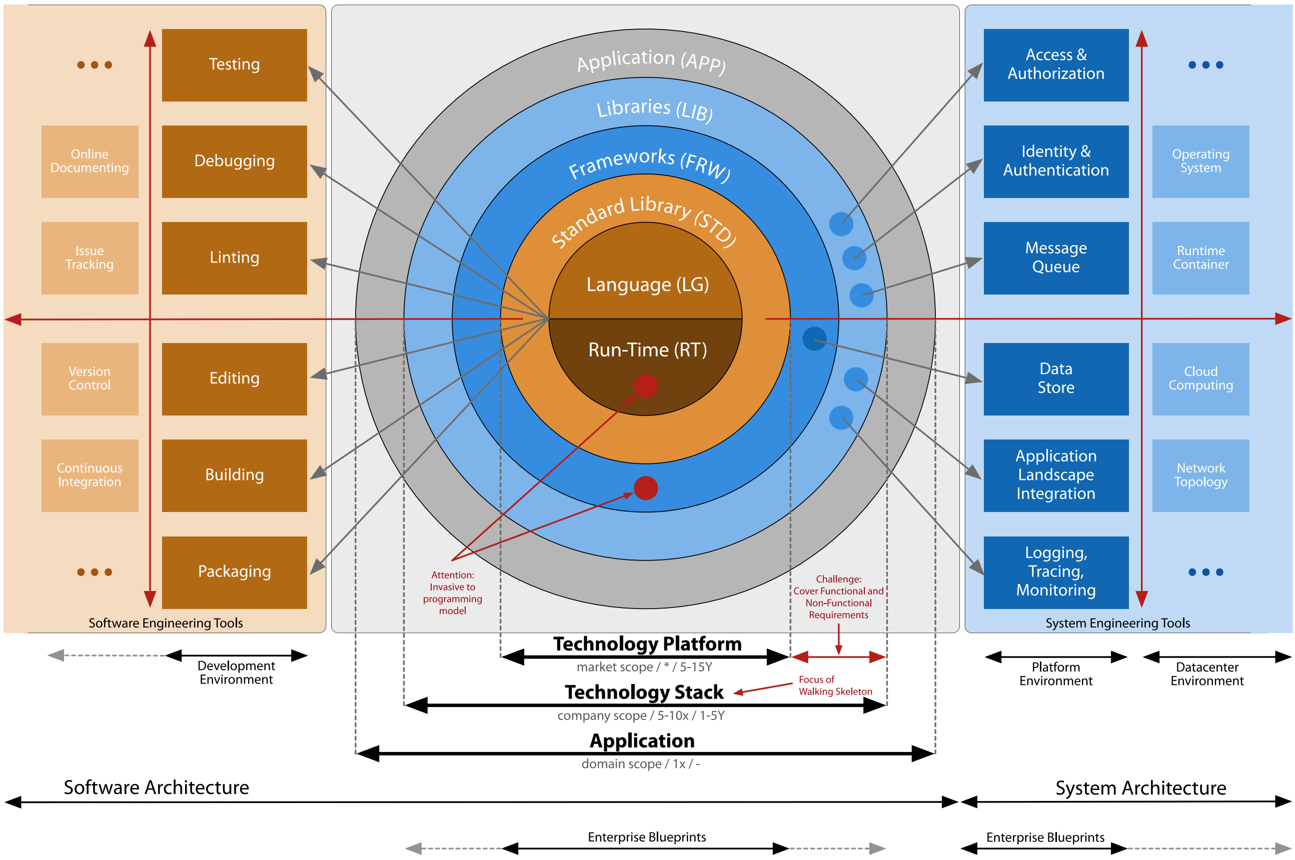
Es gibt verschiedene Ebenen von Software, von “low-level” und schwieriger (auf der Ebene des Betriebssystems) bis “high-level” und weniger schwierig (auf der Ebene von Business-Logik).

Bei einer Technologie-Plattform geht es weniger um die Auswahl einer bestimmten Programmiersprache und mehr um die Auswahl eines bestimmten Ökosystems, um auf eine bestimmte Art von Software abzuzielen!

Fragen

- ? Ist die Technologie-Plattform **Node.js** geeignet, um auf der Ebene des Betriebssystems ein Kernel-Subsystem zu implementieren?

Technology Stack


Testing
Debugging
Linting
Editing
Building
Packaging
Online Documenting
Issue Tracking
Version Control
Continuous Integration
• • •
• • •
Software Engineering Tools
Application (APP)
Libraries (LIB)
Frameworks (FRW)
Standard Library (STD)
Language (LG)
Run-Time (RT)
Attention: Invasive to programming model
Challenge: Cover Functional and Non-Functional Requirements
Access & Authorization
Identity & Authentication
Message Queue
Data Store
Application Landscape Integration
Logging, Tracing, Monitoring
Operating System
Runtime Container
Cloud Computing
Network Topology
• • •
• • •
System Engineering Tools
Development Environment
Technology Platform
market scope / * / 5-15Y
Technology Stack
Focus of Walking Skeleton
company scope / 5-10x / 1-5Y
Application
domain scope / 1x / -
Platform Environment
Datacenter Environment
Software Architecture
System Architecture
Enterprise Blueprints
Enterprise Blueprints

Eine **Technology Platform** besteht aus einer **Language**, einer optionalen **Run-Time**-Umgebung und einer **Standard Library**. Darauf aufsetzend, erweitern **Frameworks** und **Libraries** dies zu einem **Technology Stack**, in dem mit ihnen vor allem die Voraussetzungen zum Erzielen der funktionalen und nicht-funktionalen Anforderungen in der **Application** geschaffen werden.

Es ist zu beachten, daß die **Run-Time** und die **Frameworks** üblicherweise extrem “invasiv” für das Programmiermodell sind und somit fast nie nachträglich ausgewechselt werden können. Deshalb wird beim sogenannten **Walking Skeleton** (deutsch “Technischer Durchstich”) der Fokus vor allem auf den zu definierenden und zu integrierenden **Technology Stack** gelegt.

Während die **Application** einen fachlichen Geltungsbereich besitzt und nur ein Mal implementiert wird, wird ein konkreter **Technology Stack** üblicherweise von einer Firma definiert und dann mehrfach über einen Zeitraum von ein paar Jahren wiederverwendet.

Die unterliegende konkrete **Technology Platform** dagegen wird von einem Dritten für den Markt realisiert, wird beliebig oft wiederverwendet und muss für einen recht langen Zeitraum bestehen.

Große Firmen legen deshalb üblicherweise die zu nutzenden **Technology Platforms** und **Technology Stacks** in ihren **Enterprise Blueprints** stringent fest.

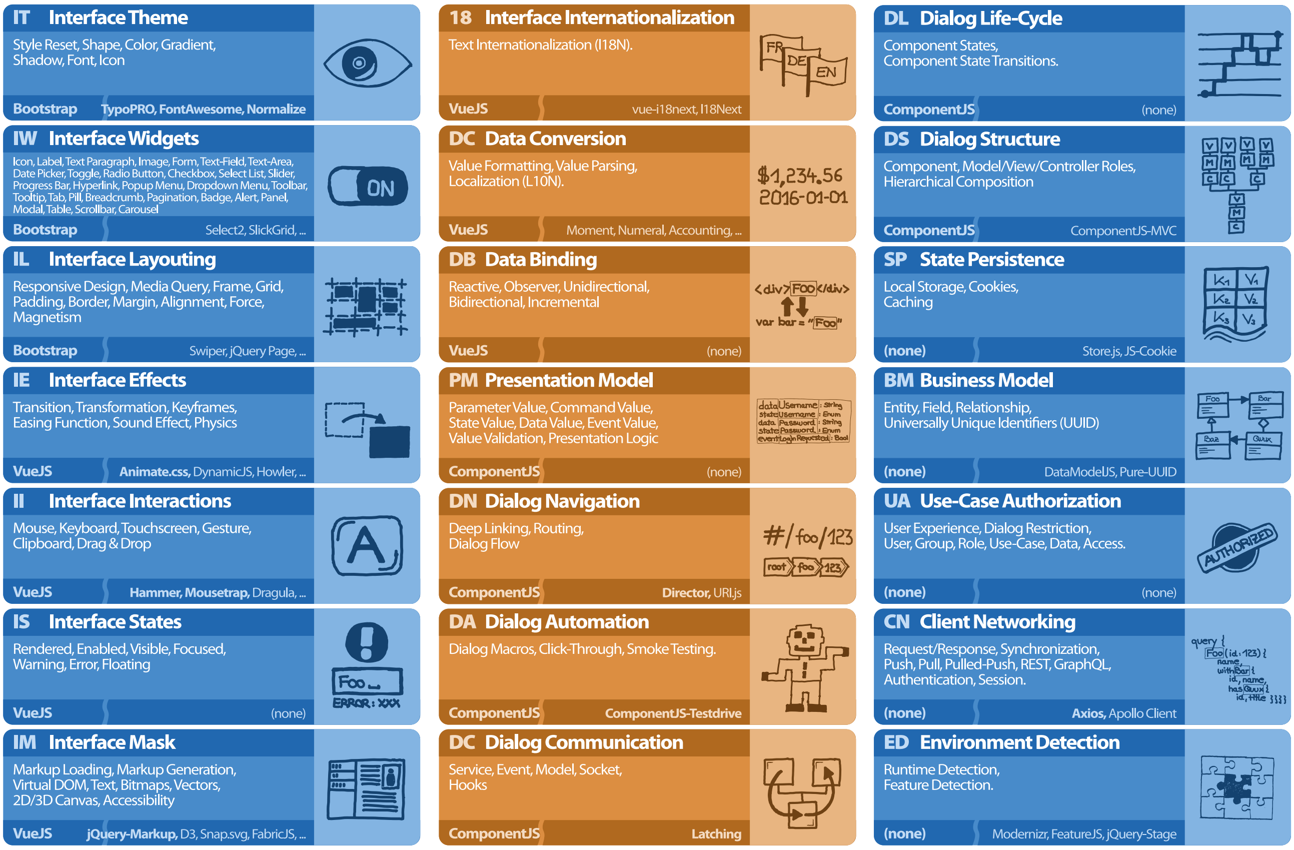
Bei den **Software Engineering Tools** sollte man bei der Definition eines Technology Stacks auch die Werkzeuge für **Testing**, **Debugging**, **Linting**, **Editing**, **Building** und **Packaging** des **Development Environments** mit berücksichtigen, denn diese sind üblicherweise direkt vom konkreten Technology Stack abhängig.

Ähnlich ist es bei den **System Engineering Tools** des **Platform Environments**: diese benötigen im **Technology Stack** mindestens zugehörige **Libraries**, um zur Laufzeit der **Application** angesprochen werden zu können.

Fragen

- ? Aus welchen drei Bestandteilen besteht eine **Technology Platform**?
- ? Aus welchen zwei zusätzlichen Bestandteilen besteht ein **Technology Stack** gegenüber der **Technology Platform**?
- ? Welche zwei Bestandteile eines **Technology Stacks** sind am “invasivsten” für das Programmiermodell?

Rich-Client Aspects



IT	**Interface Theme**	Style Reset, Shape, Color, Gradient, Shadow, Font, Icon	**Bootstrap**	**TypoPRO, FontAwesome, Normalize**
IW	**Interface Widgets**	Icon, Label, Text Paragraph, Image, Form, Text-Field, Text-Area, Date Picker, Toggle, Radio Button, Checkbox, Select List, Slider, Progress Bar, Hyperlink, Popup Menu, Dropdown Menu, Toolbar, Tooltip, Tab, Pill, Breadcrumb, Pagination, Badge, Alert, Panel, Modal, Table, Scrollbar, Carousel	**Bootstrap**	Select2, SlickGrid, ...
IL	**Interface Layouting**	Responsive Design, Media Query, Frame, Grid, Padding, Border, Margin, Alignment, Force, Magnetism	**Bootstrap**	Swiper, jQuery Page, ...
IE	**Interface Effects**	Transition, Transformation, Keyframes, Easing Function, Sound Effect, Physics	**VueJS**	**Animate.css,** DynamicJS, Howler, ...
II	**Interface Interactions**	Mouse, Keyboard, Touchscreen, Gesture, Clipboard, Drag & Drop	**VueJS**	**Hammer, Mousetrap,** Dragula, ...
IS	**Interface States**	Rendered, Enabled, Visible, Focused, Warning, Error, Floating	**VueJS**	(none)
IM	**Interface Mask**	Markup Loading, Markup Generation, Virtual DOM, Text, Bitmaps, Vectors, 2D/3D Canvas, Accessibility	**VueJS**	**jQuery-Markup,** D3, Snap.svg, FabricJS, ...
18	**Interface Internationalization**	Text Internationalization (I18N).	**VueJS**	vue-i18next, I18Next
DC	**Data Conversion**	Value Formatting, Value Parsing, Localization (L10N).	**VueJS**	Moment, Numeral, Accounting, ...
DB	**Data Binding**	Reactive, Observer, Unidirectional, Bidirectional, Incremental	**VueJS**	(none)
PM	**Presentation Model**	Parameter Value, Command Value, State Value, Data Value, Event Value, Value Validation, Presentation Logic	**ComponentJS**	(none)
DN	**Dialog Navigation**	Deep Linking, Routing, Dialog Flow	**ComponentJS**	**Director,** URI.js
DA	**Dialog Automation**	Dialog Macros, Click-Through, Smoke Testing.	**ComponentJS**	**ComponentJS-Testdrive**
DC	**Dialog Communication**	Service, Event, Model, Socket, Hooks	**ComponentJS**	**Latching**
DL	**Dialog Life-Cycle**	Component States, Component State Transitions.	**ComponentJS**	(none)
DS	**Dialog Structure**	Component, Model/View/Controller Roles, Hierarchical Composition	**ComponentJS**	ComponentJS-MVC
SP	**State Persistence**	Local Storage, Cookies, Caching	**(none)**	Store.js, JS-Cookie
BM	**Business Model**	Entity, Field, Relationship, Universally Unique Identifiers (UUID)	**(none)**	DataModelJS, Pure-UUID
UA	**Use-Case Authorization**	User Experience, Dialog Restriction, User, Group, Role, Use-Case, Data, Access.	**(none)**	(none)
CN	**Client Networking**	Request/Response, Synchronization, Push, Pull, Pulled-Push, REST, GraphQL, Authentication, Session.	**(none)**	**Axios,** Apollo Client
ED	**Environment Detection**	Runtime Detection, Feature Detection.	**(none)**	Modernizr, FeatureJS, jQuery-Stage

Um einen Technology Stack für einen **Rich-Client** zu definieren, müssen 21 **Aspects** berücksichtigt werden. Jeder Aspect wird dabei mit mindestens einem **Framework** oder einer **Library** abgedeckt. In der Praxis wird üblicherweise jeder Aspect von einem Framework und null oder mehreren Libraries abgedeckt. Das Ziel ist immer: mit einer minimalen Anzahl an Frameworks und Libraries eine möglichst große Abdeckung der Aspects zu erzielen.

Es ist ratsam, für sowohl Frameworks als auch Libraries Open Source Software (OSS) zu verwenden und nach Möglichkeit keinerlei Eigenimplementierungen, da üblicherweise sonst der Aufwand nicht im Verhältnis zum Nutzen steht. Denn bei allen Aspects handelt es sich um technische — und nicht fachliche — Aspects einer Benutzeroberfläche.

Im Falle einer Thin-Client Architecture (statt einer Rich-Client Architecture) fallen ein paar Aspects wie **Client Networking** und **Environment Detection** weg. Alle anderen Aspects sind aber weiterhin gültig, auch wenn im Fall einer Thin-Client Architecture das Frontend (und damit die Aspekte der Benutzerschnittstelle) der Application auf dem Server läuft.

Zwei wichtige Aspects behandeln das Datenmodell: das **Business Model** ist ein Datenmodell, welches direkt vom Server kommt und vom Schnitt und der Granularität exakt dem fachlichen Datenmodell des Servers entspricht. Dessen Daten werden mit einem **Presentation Model** synchronisiert, welches vom Schnitt und der Granularität exakt dem (eher technischen) Datenmodell der Benutzeroberfläche (vor allem über die Aspects **Interface Mask** und **Data Binding**) entspricht.

Fragen

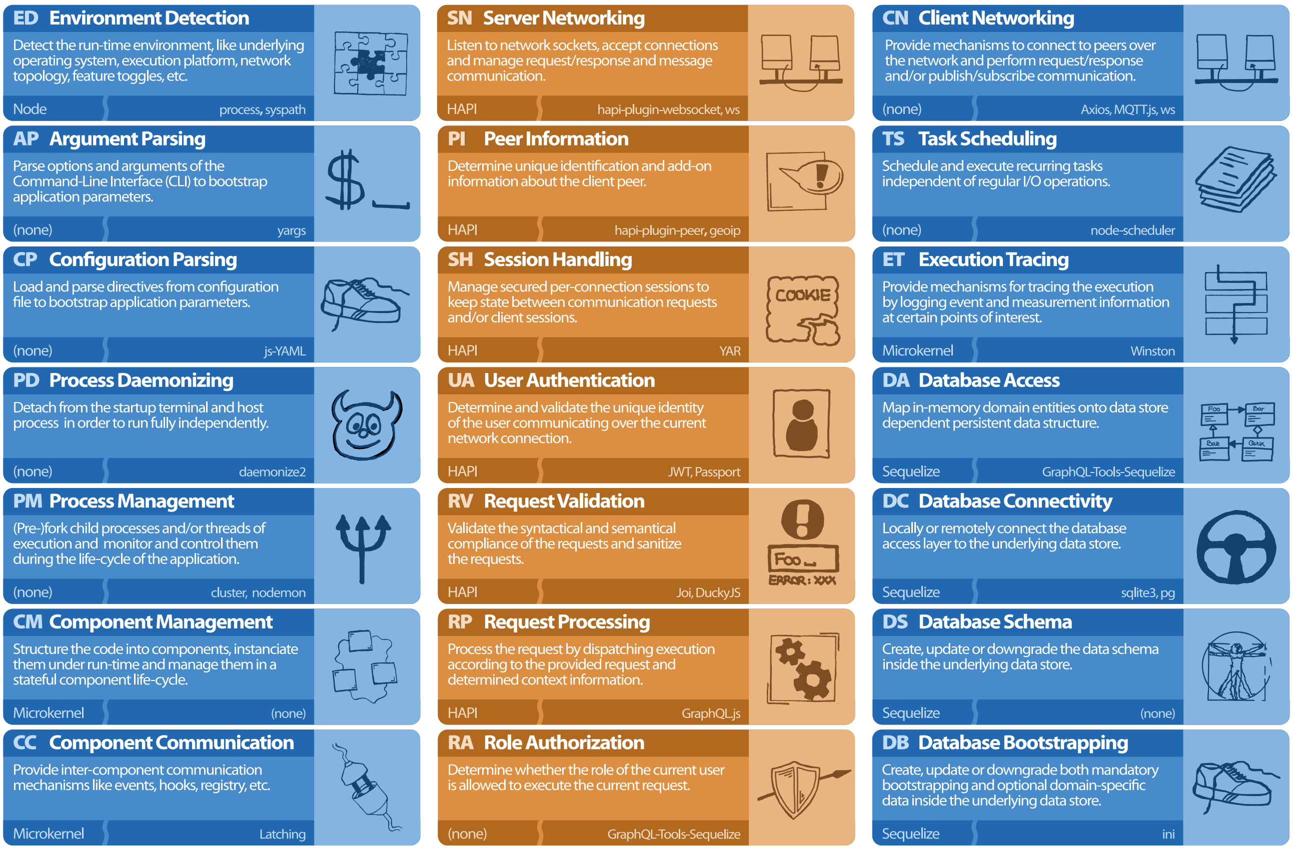
- Wie sorgt man in einer **Rich-Client Architecture** dafür, daß die zahlreichen technischen **Aspects** einer Benutzeroberfläche adressiert werden?

- Welche zwei **Aspects** einer **Rich-Client Architecture** halten das Datenmodell und kümmern sich um die Tatsache, daß man die fachlichen Daten, wie sie vom Server geliefert werden, nicht direkt in der Benutzeroberfläche verwenden kann?

ARCHITECTURE
FUNDAMENTALS

Thin-Server Aspects

TECHNISCHE
UNIVERSITÄT
MÜNCHEN



ED Environment Detection
Detect the run-time environment, like underlying operating system, execution platform, network topology, feature toggles, etc.
Node | process, syspath

AP Argument Parsing
Parse options and arguments of the Command-Line Interface (CLI) to bootstrap application parameters.
(none) | yargs

CP Configuration Parsing
Load and parse directives from configuration file to bootstrap application parameters.
(none) | js-YAML

PD Process Daemonizing
Detach from the startup terminal and host process in order to run fully independently.
(none) | daemonize2

PM Process Management
(Pre-)fork child processes and/or threads of execution and monitor and control them during the life-cycle of the application.
(none) | cluster, nodemon

CM Component Management
Structure the code into components, instanciate them under run-time and manage them in a stateful component life-cycle.
Microkernel | (none)

CC Component Communication
Provide inter-component communication mechanisms like events, hooks, registry, etc.
Microkernel | Latching

SN Server Networking
Listen to network sockets, accept connections and manage request/response and message communication.
HAPI | hapi-plugin-websocket, ws

PI Peer Information
Determine unique identification and add-on information about the client peer.
HAPI | hapi-plugin-peer, geoip

SH Session Handling
Manage secured per-connection sessions to keep state between communication requests and/or client sessions.
HAPI | YAR

UA User Authentication
Determine and validate the unique identity of the user communicating over the current network connection.
HAPI | JWT, Passport

RV Request Validation
Validate the syntactical and semantical compliance of the requests and sanitize the requests.
HAPI | Joi, DuckyJS

RP Request Processing
Process the request by dispatching execution according to the provided request and determined context information.
HAPI | GraphQL.js

RA Role Authorization
Determine whether the role of the current user is allowed to execute the current request.
(none) | GraphQL-Tools-Sequelize

CN Client Networking
Provide mechanisms to connect to peers over the network and perform request/response and/or publish/subscribe communication.
(none) | Axios, MQTT.js, ws

TS Task Scheduling
Schedule and execute recurring tasks independent of regular I/O operations.
(none) | node-scheduler

ET Execution Tracing
Provide mechanisms for tracing the execution by logging event and measurement information at certain points of interest.
Microkernel | Winston

DA Database Access
Map in-memory domain entities onto data store dependent persistent data structure.
Sequelize | GraphQL-Tools-Sequelize

DC Database Connectivity
Locally or remotely connect the database access layer to the underlying data store.
Sequelize | sqlite3, pg

DS Database Schema
Create, update or downgrade the data schema inside the underlying data store.
Sequelize | (none)

DB Database Bootstrapping
Create, update or downgrade both mandatory bootstrapping and optional domain-specific data inside the underlying data store.
Sequelize | ini

Um einen Technology Stack für einen **(Thin-)Server** zu definieren, müssen 21 **Aspects** berücksichtigt werden. Jeder Aspect wird dabei mit mindestens einem **Framework** oder einer **Library** abgedeckt. In der Praxis wird üblicherweise jeder Aspect von einem Framework und null oder mehreren Libraries abgedeckt. Das Ziel ist immer: mit einer minimalen Anzahl an Frameworks und Libraries eine möglichst große Abdeckung der Aspects zu erzielen.

Es ist ratsam, für sowohl Frameworks als auch Libraries Open Source Software (OSS) zu verwenden und nach Möglichkeit keinerlei Eigenimplementierungen, da üblicherweise sonst der Aufwand nicht im Verhältnis zum Nutzen steht. Denn bei allen Aspects handelt es sich um technische — und nicht fachliche — Aspects eines Servers.

Es ist zu beachten, daß ein Server üblicherweise nicht nur den Aspect **Server Networking** (für die Anbindung der Rich-Clients) besitzt, sondern auch den Aspect **Client Networking**, um selbst andere Server abfragen zu können.

Außerdem ist zu beachten, daß vor allem zwei wichtige Aspects den Themenkomplex Security adressieren: der Aspect **User Authentication** identifiziert und authentifiziert den Benutzer (“Ist der Benutzer derjenige?”). Der Aspect **Role Authorization** dagegen überprüft vor allen fachlichen Vorgängen, ob der authentifizierte Benutzer aufgrund seiner Rolle(n) auch wirklich autorisiert ist, die Vorgänge auszulösen (“Darf der Benutzer das?”).

Fragen

- ? Wieso besitzt ein **(Thin-)Server** üblicherweise neben dem offensichtlichen Aspect **Server Networking** auch den Aspect **Client Networking**?
- ? Welcher Aspect eines **(Thin-)Servers** kümmert sich um die Frage “Ist der Benutzer derjenige”?
- ? Welcher Aspect eines **(Thin-)Servers** kümmert sich um die Frage “Darf der Benutzer das”?